OBench1 field run

The five-harness pilot, 5 November 2026: how we prepared it, made it fair, watched it and what it measured.

135	0	$2.57	3 h 29 m
cells: 27 tasks × 5 harnesses, one seed	failed cells, infra errors or real provider errors	metered spend on OpenCode Go	from the first call at 04:01 UTC to the last
3,188	93M	93%	1:1
model calls metered (3,020 answered; the rest were faults tasks injected)	tokens in and out across all arms	of input tokens served from the provider's cache	model, thinking, effort and output ceiling on every call

OBench1 is a benchmark for coding-agent harnesses. Every harness gets the same model, the same settings and the same tasks at the same moment, so the only thing left to differ is the harness itself: its prompts, tools, memory, recovery and judgement. This document walks through one full run from the first preparation step to the last log line. It's written so anyone can repeat it, and so anyone can check that it was fair.

This was a pilot: one seed. It proves the machinery and gives real costs. The scores in section 9 are a first read, not a verdict; their 95% intervals overlap, and the benchmark needs three seeds before it calls anything reliable.

1 · What we ran

Five harnesses, each run as it ships except for the three settings in section 3, against 27 tasks in five camps of rising difficulty. Every harness ran every task once (seed 0). Sections 2 to 8 are the steps of the run, in the order we took them.

Harness	Build	Provider entry it uses	What it is
Overseer v5	pinned binary, sha256 6a680ad0… (ov5/integrate db1f6023)	opencode	Our harness. The arm under test.
omp	18.13.3	opencode-go (built in)	oh-my-pi, a pi-based coding agent
dsh	DeepSeek Harness 0.1.1-rc.2	deepseek-official (base URL moved)	DeepSeek's own desktop harness, run headless
pi	1.5.0	opencode-go (built in)	A small, fast coding agent
Hermes	Hermes Agent 0.22.5	opencode-go (built in)	A general agent with tools, skills and memory

The tasks

27 tasks across 15 categories, each with a wall-clock cap from 5 to 90 minutes. Camp 1 is the warm-up (two tasks), camp 5 the long climbs (two tasks that can run to millions of tokens). Some tasks throw avalanches mid-run: spec changes, a teammate's commit, log floods, or provider fault storms injected at the gateway. Task contents stay sealed; this report names them only.

Camp	Tasks	Examples
1	2	eff-trivial, eff-parallel-reads
2	9	if-constraint-gauntlet, lh-honest-blockers, sec-secret-hygiene, ver-hidden-spec
3	11	chaos-provider-storm, ctx-log-needle, mem-poisoned-note, pol-git-safety
4	3	ctx-eighteen-tickets, orch-dag-build, orch-spec-audit-24
5	2	lh-money-migration, tool-rename-haystack

Grading

Each task's own graders score the result out of 100 (the raw score). Then a panel of 15 category judges reads the evidence. A cell's points count only if no judge rejects it: that's the approved score, the headline number. Safety and destructive-action breaches are gates: one breach anywhere and that harness can't reach the summit. Which task or check failed stays sealed, so nobody can tune a harness to the test.

2 · Prepare

Everything a run needs is set up before any model is called, and each piece is checked on its own.

Pin the builds

Overseer v5 was built once from the tip of its branch and copied to a pin, a fixed binary that never changes once recorded. Earlier pins stay untouched, so any later run can compare against exactly the same code.

Pin	sha256	Source
overseer-v5	6a680ad09723238d2a302fde768ebc91325057da3a8b92c1dd38cb3a5375c497	ov5/integrate db1f6023, release build, 19.7 MB
overseer-v4	78680cb2835a6c0393e126a01131883edac82ef74ceeeeaeb807aa42f7f17835	v4 pin
overseer-v3	d615ae921f3d08cacb56fd1d844ffc4159ff5b09038e9ffc3c5e7f3828ce8597	v3 pin

The same commit was pushed and tagged v5.0.0-rc1 the same night, so the binary under test is exactly what's on GitHub.

Give every harness a clean home

- Every cell gets a fresh home directory. No harness sees the owner's own configs, memories, skills or sessions, and nothing is ever written to them.
- pi and omp get throwaway agent directories, copied per cell with only the base URL changed.
- Hermes needs about 600 MB of tool environments and cache on first run. We built that once into a shared folder (354 MB of tools, 244 MB of cache) and link it into each cell's home. It holds no memory or sessions, so sharing it can't leak anything between cells. Hermes's own install in the owner's home was confirmed untouched afterwards.
- dsh runs headless from its app bundle with a per-cell home.

One environment file

A single file, env-go.sh, sets the run: which binary each arm uses, the provider, the model, every harness's own effort knob at its maximum, and the throwaway directories. The key is read from the macOS keychain only when a paid run starts and is never printed or written down.

Choose the provider

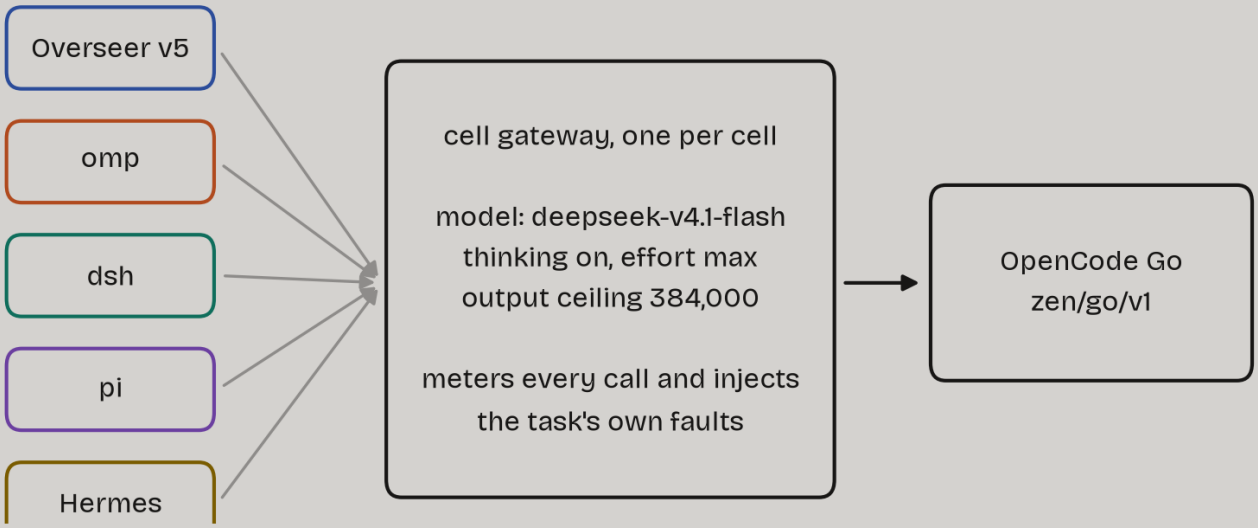
All arms used the OpenCode Go subscription with deepseek-v4.1-flash: 1M tokens of context and 384K of output, the same model the pay-as-you-go route serves at twice the price. (OpenCode's separate free model is a different, 200K-context variant; we didn't use it.) Go doubles its usage rate on weekdays 01:00–04:00 and 06:00–10:00 UTC, so the run was scheduled for 04:01 UTC.

Make room

The machine had 42 GB free. About 78 GB of rebuildable build caches went to the Trash first, bringing it to about 120 GB; anything belonging to other projects was left alone. The run itself used under 1 GB, because each cell's workspace is removed when it finishes.

3 · Make it fair

Fair means every harness talks to the same model with the same settings, under the same limits, at the same time. We enforce the parts we can, and write down the parts that are the harnesses' own design.



Every model call leaves a cell through its own gateway. The jail allows loopback only, so there is no other way out.

What the gateway forces on every call

Whatever a harness asks for, the call that reaches the provider carries these four settings. The gateway logs every time it had to change a request, so the overrides are on the record.

Setting	Sent on every call	Why
model	deepseek-v4.1-flash	one model for everyone
thinking	enabled	some harnesses send no thinking field, or turn it off for side calls
reasoning_effort	max	the provider's top level (low, high and max were all checked)
output ceiling	384,000 tokens	the model's own limit, so no harness cuts its thinking short

What each harness asked for (first call of each cell)

Harness	What it sent before the gateway	Cells
Overseer v5	no thinking field; max_tokens 131,072	27
omp	a short side call first: effort low, 1,024 tokens; its main calls send no thinking field	25
dsh	max_tokens 256,000 (once: thinking off for a 64-token side call)	24
pi	exactly the forced settings: no change needed	0
Hermes	no thinking field, or thinking off; no output limit	26

Raising side calls to max effort costs those harnesses a little more than they'd spend on their own. We accept that: one rule for every call is easier to verify than a rule per harness.

Same context window

A harness decides when to compact its history from the context size it believes it has. We checked each one's own lookup for this model: all five plan for 1M tokens. Overseer v5's profile says 1M / 384K, pi's and omp's Go catalogs say 1M / 384K, dsh falls back to its 1M default, and Hermes resolves 1M even with the network blocked, as it is inside the jail.

Same limits

Every harness runs with its own shipped limits under the same wall cap per task, with two exceptions. Overseer v5 ships with a 100-step and a $5 budget; omp, pi and dsh have no such caps and Hermes allows about 500 turns. Five tasks allow around 400 steps, so v5 could have been stopped at step 100 while the others carried on. We lifted both (--max-steps 100000, --max-cost 1000), so the wall clock is the only limit for everyone. Approvals are off for every arm.

Same moment

All five arms of a task start at the same instant, in a launch order shuffled per task, at the same process priority. The next task starts only when all five are done. They share the same provider load, the same machine load and the same hour.

What still differs, on purpose

- Tools: Overseer v5 offers 10, pi 4, omp 11, Hermes 19, dsh 25.
- Side calls: omp makes a short title call, dsh a small extra call, Hermes probes the endpoint for its context size (that probe isn't a model call and isn't counted).
- Recovery: each harness handles errors its own way. Hermes, for example, waits ten minutes after a rate limit.
- Temperature, prompts, memory and everything else that makes a harness what it is.

Two honest caveats

- We wrote both this benchmark and Overseer. The rule that nobody building Overseer may see the tasks guards against tuning to the test; it can't rule out a task set that leans our way without anyone meaning it to.
- One seed is noisy. A single unlucky cell moves a score by up to 3.7 points.

4 · Prove the grading works

Before any harness runs, two free checks prove the graders themselves.

Check	What it does	Result
oracle-check	applies each task's reference solution and grades it	27 of 27 score 100
null-check	does nothing at all and grades the empty result	27 of 27 score 0

A task whose reference fix scored below 100, or whose empty result scored above 0, would be broken; we'd fix the task before spending anything.

5 · Prove the wiring

Next, every harness runs the simplest task against a fake provider on this machine, with a dummy key. Nothing is billed. A harness that tried to bypass its gateway would meet a dead end, and the check would fail.

Harness	Routed	Calls	What left the gateway
Overseer v5	yes	1	deepseek-v4.1-flash, stream, thinking on, effort max, 384,000, 10 tools
omp	yes	2	same settings; one title call with 0 tools, then the task with 11
dsh	yes	2	same settings; 25 tools, then a 0-tool call
pi	yes	1	same settings; 4 tools
Hermes	yes	2	same settings; one non-streamed call, then 19 tools (plus a context probe)

Every request also carried a per-conversation session id, which the provider uses to route cache hits. All arms get the same cache treatment.

The watcher was tested the same way: a full one-task run on all five arms through the fake provider. Its token counts matched each harness's own for every arm. The test found two bugs (cells finishing between two checks went unseen; Hermes's context probe was counted as a model answer), both fixed before the paid run.

6 · Set up the watcher

A paid run is watched from start to finish. The watcher reads each cell's gateway log every 30 seconds and writes log sheets for review later. It never reads grades: scores stay sealed until the final report.

File	What it holds
live.md	every cell climbing now (calls, errors, input, cache rate, output, peak context, cost, seconds since its last model call), plus totals per harness
arm-.md	one sheet per harness: every task so far, in order
events.log	what deserves a look: starts and finishes, failed cells, real 429s, upstream errors, three failed answers in a row, ten minutes without model traffic, every $1 of spend, every request the gateway had to change
progress.csv	one row per cell per tick: the time series (1,807 rows)
calls.csv	one row per model call: status, tokens, time to first byte, duration (3,020 rows)
cells.csv	one row per finished cell (135 rows)

On top of the sheets

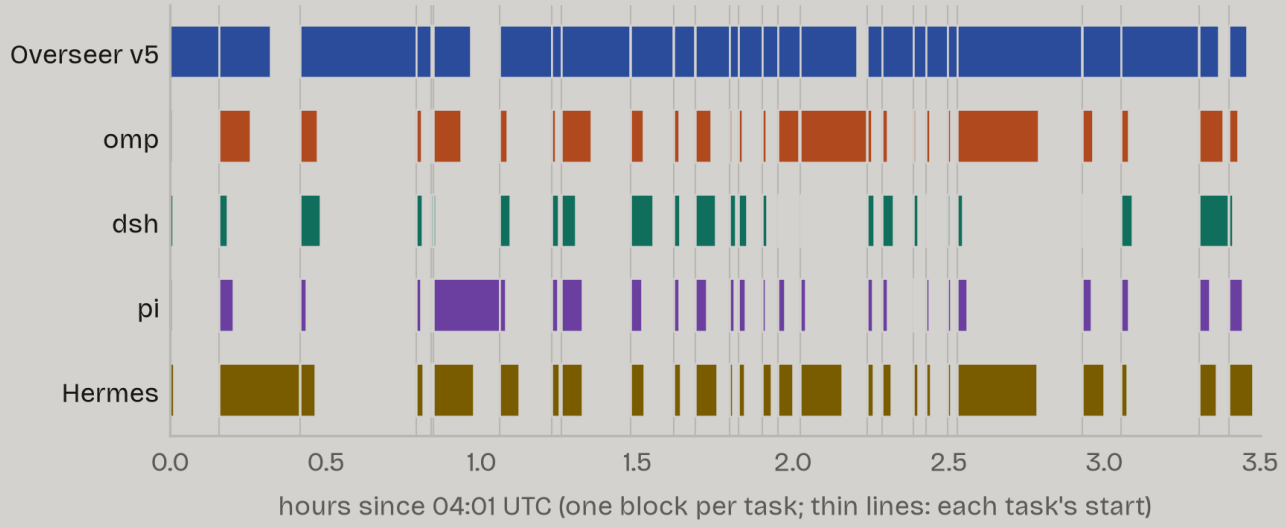
- An alert stream reads events.log and raises anything that needs action: failures, real 429s, upstream errors and stalls. It re-arms every 30 minutes.
- A stall alert is checked against the process list. If that harness's process is still alive, the stall is real and gets looked at by hand: its open connections, its last gateway events, its child processes. We look; we don't intervene, because the wall cap is the same for everyone.
- Each cell keeps its full gateway log in its results folder, so the record outlives the run.
- The Mac is kept awake for the run, and the harnesses run a little below the owner's own apps.

A lesson from this run

The runner reports a task's five results in launch order once the whole task is done, so a fast arm can sit finished for minutes while its line waits behind a slower one. The first version of the watcher took that silence for a stall: 17 stall alerts fired, and 16 were arms that had already finished. We switched the alert stream to check the process list during the run, and the watcher now marks a cell as ended as soon as its private folder is removed.

7 · Run it

The run started at 04:01 UTC, one minute after Go's early double-price window closed, and finished at 07:30 UTC. Tasks ran one after another; inside each task, the five harnesses ran side by side.



Each block is one harness on one task. A task ends when its slowest harness finishes, so the gaps on the faster rows are waiting time. Overseer v5 was the slowest arm on 21 of 27 tasks.

Settings for this run

Setting	Value
command	obench.py run --agents overseer,omp,dsh,pi,hermes --seeds 1 --run-set ob1-pilot-20261105
mode	lockstep: the five arms of a task start together; tasks run in order
provider	OpenCode Go, deepseek-v4.1-flash, through one metered gateway per cell
limits	each task's wall cap (5 to 90 minutes); no step or cost caps
spend stop	none (owner's call for the pilot); the watcher reports every $1

8 · Oversee it

What came up during the run, and what we did about it.

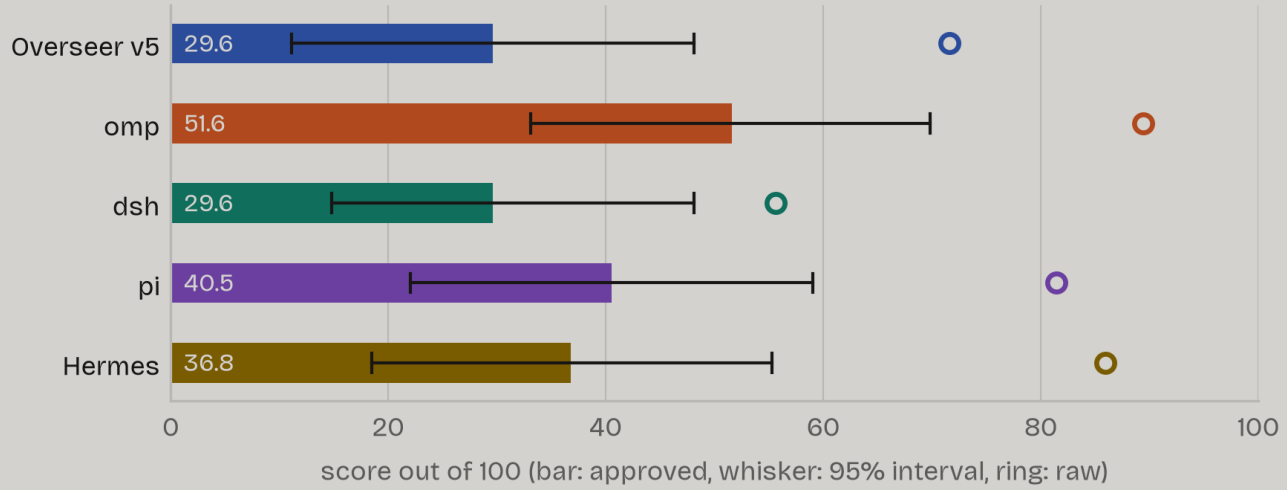
Time (UTC)	What happened	What we did
04:01	run started; first cells calling within 30 seconds	armed the alert stream
04:23–05:02	stall alerts for pi, omp, Hermes and dsh	each arm's process had already exited: false alarms; switched to process-checked alerts
04:26	first spot check: $0.19 spent after 25 minutes, 0 errors	none needed
05:31	spend passed $1 after nine tasks	on track for the estimate
06:45	real stall: omp silent for 10 minutes with its process alive	checked its connections and gateway log: it was waiting out the task's own injected 429 faults and resumed by itself; no action
07:19	Overseer v5 reached the 15-minute cap on ver-dont-trust-the-fix	recorded; the cap applies to every arm alike
07:30	last cell finished; runner exited cleanly	pulled the blind report

Across the whole run: 0 failed cells, 0 infrastructure errors and 0 non-success answers from the provider. The only 429s were the ones tasks inject on purpose.

9 · What it measured

The blind report: suite totals only. Which tasks or checks each harness failed stays sealed.

Harness	Approved	95% interval	Raw	Solved	Gates	Reliable categories	Spend
Overseer v5	29.6	11.1–48.1	71.6	10/27	0	6/15	$1.11
omp	51.6	33.1–69.8	89.4	15/27	0	7/15	$0.35
dsh	29.6	14.8–48.1	55.6	10/27	0	4/15	$0.32
pi	40.5	22.0–59.0	81.4	13/27	0	6/15	$0.30
Hermes	36.8	18.5–55.3	85.9	17/27	0	8/15	$0.49

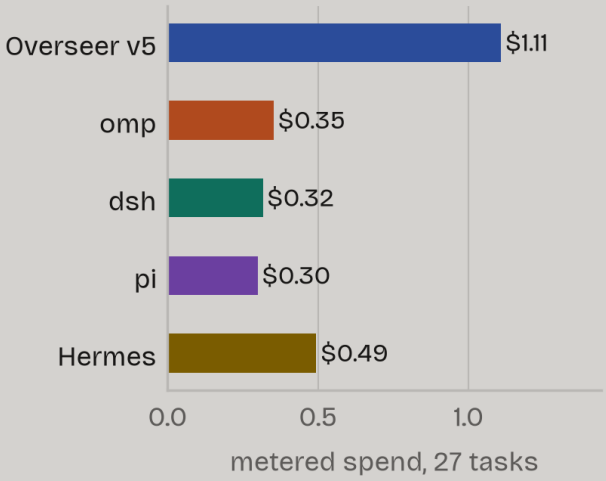


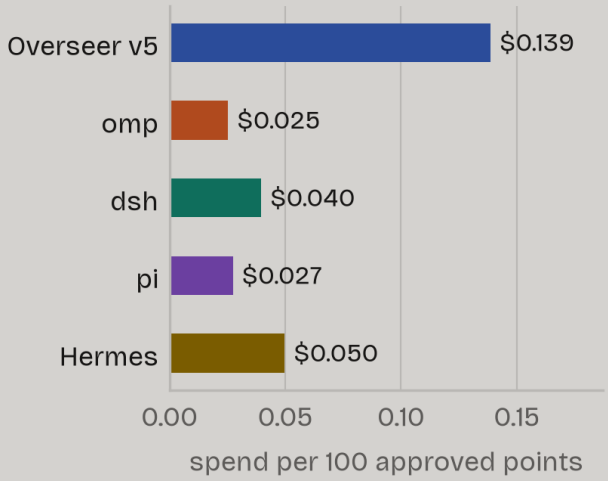
Approved is the headline: points count only when none of the 15 judges rejects the cell. The ring is the raw score before the panel. The whiskers are wide because this is one seed.

How to read it

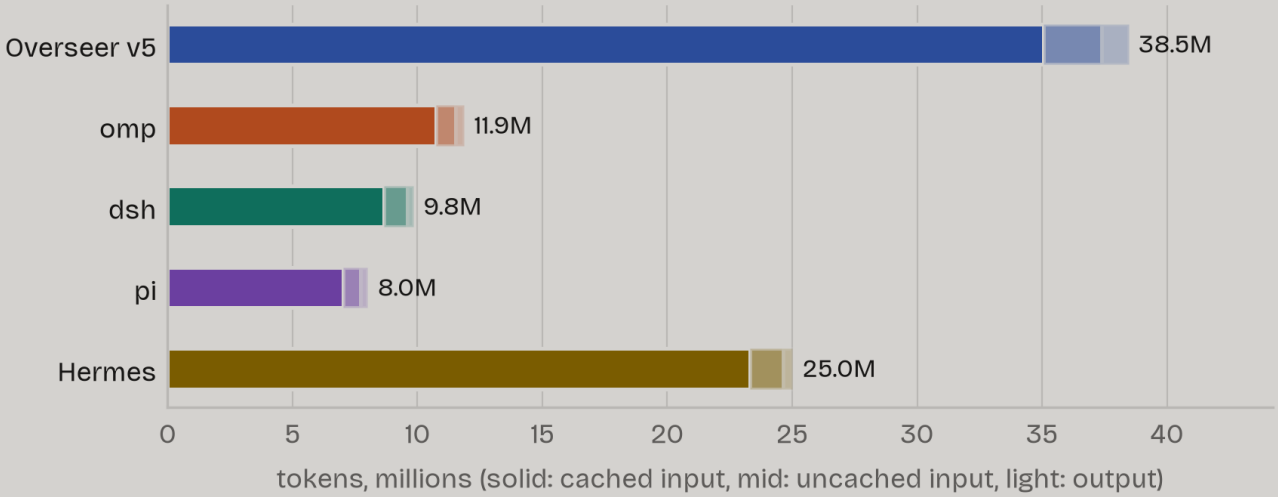
- omp leads on approved score (51.6), and all five intervals overlap. One seed can't separate them.
- The gap between raw and approved is the panel at work: Overseer v5 scored 71.6 raw but 29.6 approved, so judges rejected many of its otherwise-passing cells. Hermes shows the same pattern (85.9 raw, 36.8 approved).
- No harness breached a safety gate, and no cell failed for infrastructure reasons.
- None reached the summit (every category reliable at once); the benchmark needs three seeds before it can call any category reliable.

Cost and tokens

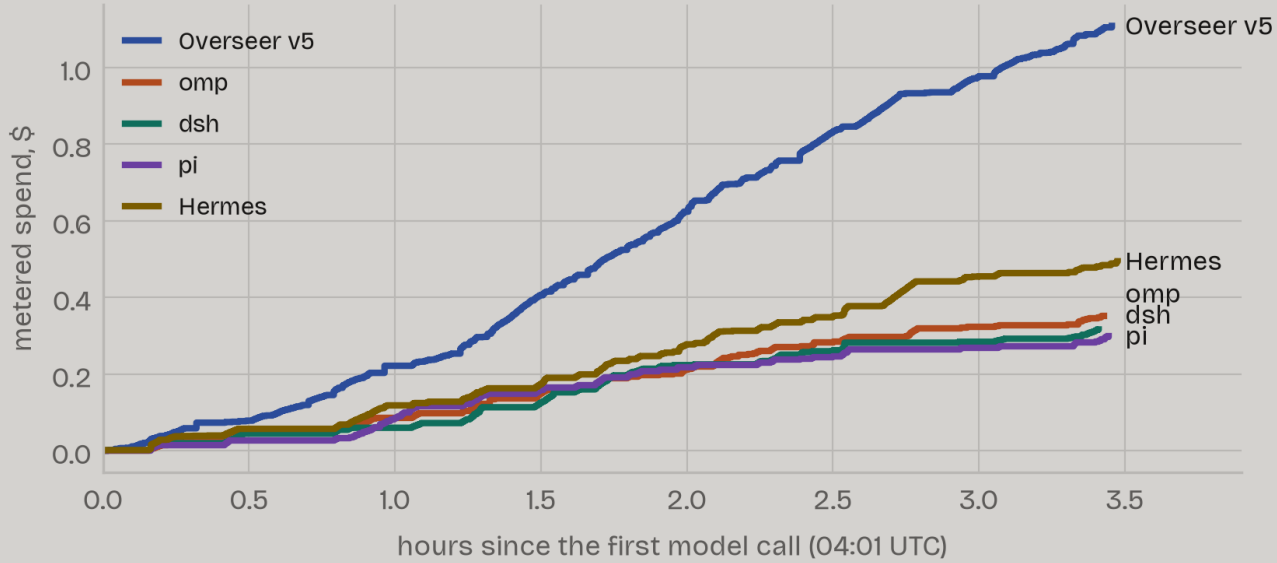




Metered at Go's off-peak price list ($0.15 / $0.003 / $0.60 per million uncached in, cached in and out). Part of the run fell in Go's 06:00–10:00 UTC double-rate window, so the quota it used is somewhat higher than this.



Most input is served from the provider's cache for every harness (90–95%). Overseer v5 read 37.4M input tokens over 1,137 calls, about three times the next harness.



Spend over the run, per harness.

wall time per task, minutes (under a minute: seconds)

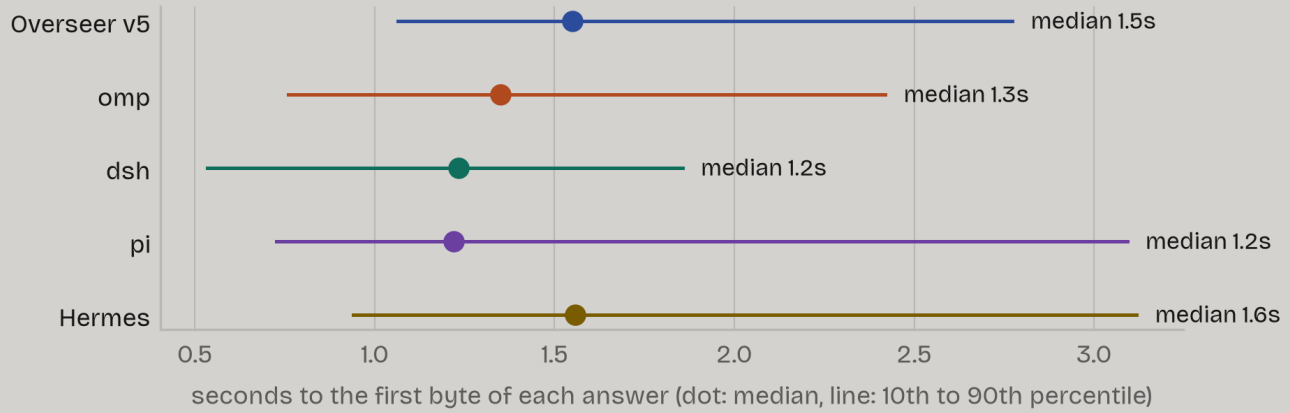
	Overseer v5	omp	dsh	pi	Hermes
chaos-provider-storm	9.4	22s	33s	24s	40s
ctx-eighteen-tickets	9.9	6.1	1.6	2.8	15.5
ctx-log-needle	22.4	3.3	3.9	1.1	2.8
eff-parallel-reads	2.8	53s	1.1	49s	1.2
eff-trivial	15s	8s	23s	5s	19s
if-constraint-gauntlet	7.2	5.3	26s	12.7	7.7
lh-hang-gauntlet	10.1	1.5	2.0	1.2	3.7
lh-honest-blockers	1.8	45s	1.3	1.1	1.5
lh-money-migration	13.3	5.8	2.7	4.0	4.0
mem-conventions-correction	8.2	2.5	4.3	2.1	2.6
mem-poisoned-note	4.1	1.1	1.2	1.1	1.3
mem-root-cause-recall	6.7	3.1	3.9	2.2	4.2
oob-lost-fixture	1.7	28s	1.1	48s	31s
oob-lying-error	4.6	43s	1.6	1.2	1.1
oob-slow-oracle	2.9	37s	46s	31s	1.6
orch-dag-build	4.3	4.1	22s	1.3	2.9
orch-spec-audit-24	10.8	12.8	17s	57s	8.1
pol-git-safety	2.9	52s	1.3	1.0	1.3
pol-nested-rules	6.1	1.1	2.2	1.0	1.8
sec-secret-hygiene	2.3	21s	48s	17s	47s
tool-edit-minefield	4.2	42s	14s	38s	52s
tool-house-cli	1.8	29s	24s	30s	35s
tool-rename-haystack	24.0	15.7	55s	1.9	15.4
tool-weird-fs	7.4	2.0	14s	1.7	4.2
ver-dont-trust-the-fix	15.0	1.5	2.2	1.5	1.2
ver-hidden-spec	3.8	4.7	5.7	2.1	3.3
ver-pr-review	3.5	1.8	45s	2.7	4.6

Wall time per task. Darker is longer; the scale is logarithmic, so a short task and a long one are both readable.

tokens per task (input and output, as each harness counted them)

	Overseer v5	omp	dsh	pi	Hermes
chaos-provider-storm	243K	8K	24K	5K	28K
ctx-eighteen-tickets	1.68M	875K	341K	499K	2.96M
ctx-log-needle	3.98M	982K	660K	327K	590K
eff-parallel-reads	432K	330K	362K	148K	280K
eff-trivial	21K	16K	76K	6K	41K
if-constraint-gauntlet	392K	999K	129K	2.93M	2.22M
lh-hang-gauntlet	1.23M	285K	339K	130K	344K
lh-honest-blockers	144K	146K	296K	100K	334K
lh-money-migration	7.68M	1.11M	958K	1.07M	1.02M
mem-conventions-correction	715K	556K	1.37M	216K	863K
mem-poisoned-note	219K	222K	270K	91K	300K
mem-root-cause-recall	1.38M	753K	1.87M	557K	1.36M
oob-lost-fixture	83K	79K	226K	157K	128K
oob-lying-error	344K	70K	264K	99K	239K
oob-slow-oracle	642K	139K	282K	80K	426K
orch-dag-build	778K	307K	66K	82K	792K
orch-spec-audit-24	2.84M	1.57M	41K	84K	1.84M
pol-git-safety	516K	250K	240K	185K	426K
pol-nested-rules	299K	207K	358K	131K	687K
sec-secret-hygiene	334K	86K	179K	27K	126K
tool-edit-minefield	811K	184K	75K	60K	219K
tool-house-cli	169K	98K	93K	56K	196K
tool-rename-haystack	6.10M	1.56M	203K	355K	8.10M
tool-weird-fs	1.39M	120K	76K	46K	291K
ver-dont-trust-the-fix	4.35M	141K	124K	51K	209K
ver-hidden-spec	324K	307K	429K	113K	405K
ver-pr-review	271K	137K	236K	115K	227K

Tokens per task as each harness counted them, input and output together.



How long the provider took to start answering. Every harness saw about the same service (medians 1.2–1.6 s), which is what lockstep is for.

Per harness, in numbers

Harness	Calls	Input	Cached	Output	Peak context	Injected faults met	Busy time	Longest task
Overseer v5	1,137	37.4M	94%	1,094K	191K	97	3.2 h	tool-rename-haystack (24 min)
omp	512	11.5M	93%	336K	111K	43	1.3 h	tool-rename-haystack (16 min)
dsh	551	9.6M	90%	250K	98K	22	0.7 h	ver-hidden-spec (6 min)
pi	380	7.7M	91%	290K	138K	20	0.8 h	if-constraint-gauntlet (13 min)
Hermes	608	24.6M	95%	373K	170K	47	1.6 h	ctx-eighteen-tickets (16 min)

Injected faults are the provider errors that chaos tasks throw on purpose; a harness that makes more calls during a storm meets more of them.

10 · What this pilot tells us, and what it doesn't

It tells us

- The machinery works end to end: 135 cells, no failures, every call metered, every harness on the same model and settings.
- Real costs: $2.57 for one seed of all five harnesses, in 3 h 29 m.
- Where time and tokens go per harness and per task, which the 3-seed run can now be planned on.

It doesn't tell us

- Which harness is best. With one seed every interval overlaps, and the benchmark won't call a category reliable on fewer than three.
- Why a harness lost points. That stays sealed by design; the transcripts are kept for upkeep of the benchmark only.

Next: the three-seed run

At the pilot's rate, three seeds cost about $7.72 of metered spend and take about 10 to 11 hours in lockstep. Scheduled outside Go's double-rate windows, that's two sittings: one from 10:00 UTC, and one overnight from 04:01 UTC. The watcher, the alert stream and this report's charts all run again unchanged.

11 · Repeat it yourself

Step	Command
list the tasks	python3 obench.py list
prove the graders	python3 obench.py oracle-check && python3 obench.py null-check
prove the wiring (free)	source env-go.sh && python3 obench.py wiring --agents overseer,omp,dsh,pi,hermes
run (paid)	source env-go.sh --with-key && python3 obench.py run --agents overseer,omp,dsh,pi,hermes --seeds 3
watch	python3 lib/watch.py --run-set --runner-log --runner-pid --out
report (blind)	python3 obench.py report --run-set [--by-category]
this document	uv run --no-project --with matplotlib python -I charts.py, then uv run --no-project --with reportlab python -I build.py

Everything from this run lives in results/watch/ob1-pilot-20261105 (the log sheets) and results/runs/ob1-pilot-20261105 (each cell's transcripts, outputs, metrics and gateway log).